KNU 강원대학교
KANGWON NATIONAL UNIVERSITY

# R E S U L T  R E P O R T

# 마이크로프로세서응용및실험 결과보고서_USART




| | |
|---|---|
| 수 강 과 목 | : 마이크로프로세서응용및실험-2분반 |
| 담 당 교 수 | : 김도인 교수님 |
| 학 번 | : 202210820 |
| 작 성 자 | : 이동제 |
| 공 동 실 험 자 | : 강민수(202214221) |
| 제 출 일 | : 2024-06-20 목요일 |

# 1. 실험목적

ATmega128 마이크로컨트롤러의 USART 기능을 이용한 직렬 통신 방법을 배운다.

# 2. 실험원리

USART라는 외부 주변기기와 직렬 인터페이스를 통해 데이터를 송수신할 수 있도록 하는 장치를 통해 직렬 통신 기능을 사용할 수 있습니다. 직렬 통신은 병렬 통신과 비교하면 느리다는 단점이 있지만, 대신 먼 거리를 통신할 수 있다는 장점이 있습니다. ATmega128의 USART는 전 이중 방식으로 송신선과 수신선이 각각 두 가지로 나뉘어 있습니다.

그리고 동기화 클록을 사용하지 않는 비동기 방식이기 때문에 미리 1초에 몇 개의 데이터를 보낼지 약속해야 합니다.
1초에 몇 개의 데이터를 보내는지 나타내는 단위를 bps(bit per second)라고 부릅니다.

우리의 ATmega128은 RXD1과 TXD1이 위치한 PD2핀과 PD3핀만 연결되어 있어 이 핀들만 갖고 실습을 진행합니다.
PD2 핀에 위치한 RXD1는 R은 수신(Recieve)을 뜻하며, PD3 핀에 위치한 TXD1의 T는 송신(Transmission)을 의미합니다.
각각의 핀들은 이름이 의미하는 역할을 수행합니다.

USART의 UDR(Transmit)은 임시저장소(버퍼)의 역할을 합니다. 하드웨어적으로 송신과 수신 2개가 존재합니다.
USART는 특별하게 시작 비트+데이터 비트+패리티 비트+정지 비트로 이루어진 데이터 프레임으로 존재합니다.

Frame

| (IDLE) | St | 0 | 1 | 2 | 3 | 4 | [5] | [6] | [7] | [8] | [P] | Sp1[Sp2] | (St/IDLE) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

St은 Start의 약자로 시작 비트를 뜻합니다. 1비트를 필요로 하며, 항상 Low(=0)의 값을 가지고 있습니다. 이제 1을 집어넣으면 데이터가 들어간다고 알려주는 역할을 합니다.
0~[8]의 데이터 비트는 사용자가 보내고 싶은 데이터를 입력할 수 있으며, 대괄호의 의미는 최소 5비트에서 최대 9비트를 보낼 수 있기 때문에 대괄호부턴 선택사항이라는 의미입니다.
[P]는 데이터의 에러를 검출하는 패리티 비트로서 짝수 패리티, 홀수 패리티를 선택해서 사용하거나 사용하지 않을 수 있습니다.
Sp1[Sp2]는 들어가는 곳부터는 데이터가 끝났다고 알리는 정지 비트로서 1비트 또는 2비트를 넣어줄 수 있으며, 스타트 비트와 반대로 항상 High(=1)의 값을 가집니다.

이들을 제어하는 레지스터는 아래와 같습니다.

**1. UDR1(USART1 Data Register)** : USART1의 전송/수신 데이터 버퍼 레지스터로서 UDR을 통해 송신을 하려면 UDR에 Write를, 수신을 할 땐 UDR에서 Read합니다.

**2. UCSR1A(USART1 Control and Status Register A)** : 송수신의 동작 제어 및 상태 표시를 하는 레지스터로 수신과 송신 완료 상태를 나타내는 플래그 비트, 데이터를 받을 준비가 되면 1로 변하는 플래그 비트와 프레임 에러, 오버런 에러, 패리티 에러를 확인할 수 있습니다. 그 외에 통신속도 2배속 설정과 멀티 프로세서 통신 모드로 설정할 수 있지만 마지막 두 가지 기능은 이번 실습에서는 사용하지 않습니다.

**3. UCSR1B(USART1 Control and Status Register B)** : USART의 송수신 완료 인터럽트를 허용하거나 거부하는 기능과 통신 데이터 크기를 설정할 수 있습니다. 이 중에 데이터 크기를 설정하는 UCSZn2는 UCR1C의 UCSZn1, UCSZn0과 함께 데이터 크기를 설정합니다. 그리고 UDR의 최대 비트가 8비트인데 9비트로 쓸 수 있는 이유는 RXB8n에서 9번째 비트(MSB)를 수신하여 합쳐서 사용하게 됩니다.

**4. UCSR1C(USART1 Control and Status Register C)** : 동작 및 통신 데이터 크기를 설정하는 레지스터로 비동기와 동기 통신 모드를 고를 수 있고, 패리티 모드를 사용할지 여부와, 정지 비트의 개수를 설정할 수 있습니다. 그리고 아까 위에서 언급한 비트들이 B의 비트와 함께 데이터 크기를 설정합니다. 그리고 동기화 클럭을 사용할 때 상승 엣지로 할지, 하강 엣지로 할지도 설정할 수 있습니다.

**5. UBRR1H와 UBRR1L** : 이 레지스터는 두 개의 레지스터를 합쳐서 16개 비트 중 12개의 비트를 사용하여 USARTn의 보레이트(Baud Rate)를 설정하기 때문에 거의 하나라고 봐도 무방합니다.

우리는 UBRRn을 통해 계산하기 때문에 보레이트 계산식보단 UBRRn 계산식을 사용합니다.

$$UBRRn = \frac{f_{OSC}}{16 \times BAUD} - 1$$

이러한 레지스터를 비롯한 USART 기능의 실험 원리들을 가지고
AVR studio에서 작성한 코드와 함께 컴퓨터의 하이터 터미널을 통하여 테스트를 진행합니다.

# 3. 실험기구 및 재료

| 실험기구 및 재료 | 수량 |
|---|---|
| ATmega128 마이크로 컨트롤러 | 1개 |
| 컴퓨터 | 1개 |
| ISP 케이블 | 1개 |
| 소형 드라이버 | 1개 |
| USB-to-RS-232 어댑터 | 1개 |

표 1 ) 실험기구 및 재료

# 4. 실험방법

USART 기능을 사용하는 테스트를 진행하기 위해 하이퍼 터미널의 초기 설정을 먼저 진행합니다.

ATmega128과 USB-to-RS-232 어댑터를 연결하여 컴퓨터에 꽂습니다.

장치 관리자을 실행하여, 포트(COM & LPT)를 선택하여 ATmega128과 연결된 Prolific USB-to-Serial Comm Port(COM4 혹은 COM7, 컴퓨터에 따라 숫자가 다름)의 모델을 확인하고 같은 모델로 설정합니다.

하이퍼 터미널을 실행한 후, 이름을 입력하고 비트/초를 19200, 데이터 비트는 8, 패리티는 없음, 정지 비트는 1, 흐름 제어(F)는 없음으로 설정해줍니다.

마지막으로 파일탭의 속성-설정-ASCII 설정-입력된 문자를 터미널 창에 표시를 선택하여 활성화합니다.
이로서 하이퍼 터미널에 ASCII 코드를 띄울 준비를 마쳤습니다.

이제 추가 실습에서 원하는 값을 출력하기 위해 AVR Studio에서 코드를 작성합니다.

### 추가실습1) 수신 완료 인터럽트(Receive Complete interrupt)를 사용하여 키보드로 수신 받은 값을 하이퍼터미널로 송신하시오. (UCSR1B=0x98)

```
#include <avr/io.h>
#include <util/delay.h>
unsigned char temp;
```

I/O 레지스터와 딜레이 함수를 쓰기 위한 헤더파일 사용한다고 설정합니다.
그리고 임시변수 temp를 설정합니다.

```
int main(void)
{
        UCSR1A=0x00;
        UCSR1B=0x98;                    // RXEN,TXEN set: 송수신기 enable
        UCSR1C=0x06;
        UBRR1H=0x00;
        UBRR1L=0x33;                    // UBRR=51, Baud Rate: 16MHz일 때 19.2k
```

USART 기능을 사용하기 위해 관련 레지스터에 대한 설정을 합니다. UCSR1A는 설정하지 않고 기본 상태로 둡니다. UCSR1B 레지스터에 0x98이란 값을 넣어 수신 완료 인터럽트를 활성화하고 송수신을 활성화합니다. 그리고 송수신 데이터 비트의 크기는 8비트로 설정합니다.

그리고 UCSR1C를 설정하여 비동기 모드, 패리티 비트 사용 안 함, 정지 비트는 1, UCSR1B의 UCSZ2와 함께 문자 크기도 8비트로 맞춰줍니다. 0b0011

보레이트(bps)를 초당 19.2K로 설정하기 위해 ATmega128의 주파수인 16MHz와 비동기 일반 모드를 사용할 때의 보레이트 계산식을 사용합니다. 51에 해당하는 16진수인 UBRR1L=0x33으로 설정합니다. (16진수 계산: 3x16^1+3x16^0=51)
상위 비트인 UBRR1H를 쓸 정도 크기의 숫자가 아니기 때문에 0x00;으로 비워둡니다.

$UBRRn = \frac{f_{OSC}}{16 \times BAUD} - 1$ 의 계산식을 사용하면 $\frac{16 \times 10^6}{16 \times 19.2 \times 10^3} - 1 = 51$

```
        while(1)
        {
                if((UCSR1A&0x80)==0x80) {          // 수신 데이터 여부 check
```

while 문을 통해 다음 함수들이 이 안에서 무한루프하게 합니다.
if문을 통해 UCSR1A 레지스터의 RXC1 비트가 1이면 수신 데이터가 존재한다는 것으로, 수신 데이터가 존재할 때마다 if문이 작동하도록 합니다.

```
                temp=UDR1;                         // 수신 데이터를 임시변수에 저장
                while((UCSR1A&0x20)==0x00);        // 송신 가능 check
                UDR1=temp;                         // 수신 문자를 송신
                }
        }
}
```

데이터가 수신되는 장소인 UDR1에서 리드하여 변수 temp에 저장되도록 합니다.
두 번째 while문에는 조건을 달아서 UCSR1A 레지스터의 데이터 레지스터가 비어있어 송신 준비가 된지 확인 후, temp 변수에 저장되었던 값을 UDR1에 다시 보내어 수신되었던 문자를 송신시킵니다.

### 추가실습2) 하이퍼터미널에 문자열을 출력하시오.

```
#include <avr/io.h>
#include <util/delay.h>

unsigned char Tmp[16]={'K','N','U','_','c','j',' ','J','e','o','n','g','_','P','E','L'};
int cnt =0;
```

이번엔 "KNU_cj Jeong_PEL"이라는 16개의 문자를 사용하기 위해 Tmp를 문자 배열 변수로 사용합니다.
그리고 변수 cnt에 0을 집어넣어서 사용한다고 선언합니다.

```
int main(void)
{
        UCSR1A=0x00;
```

```
        UCSR1B=0x08;                // TXEN set: 송신기 enable, PD0=RxD
        UCSR1C=0x06;                // Asynchronous operation, no parity, stop_bit 1,
        UBRR1H=0x00;
        UBRR1L=0x33;                //  UBRR=51 ? 12bit baud rate(19.2k at 16MHz)
```

UCSR1B는 수신은 비활성화, 송신을 활성화하는 비트를 제외하고 인터럽트 관련 비트는 모두 비활성화합니다. 나머지는 추가실습1에서 설정했던 값과 같습니다.

```
                for(cnt=0;cnt<16;cnt++)
                {
                        while((UCSR1A&0x20)==0x00);          // 송신버퍼 UDR에 쓸 수 있는지 확인
                        _delay_ms(2000);
                        UDR1=Tmp[cnt];}                      //  문자 배열 출력
                }
        while(1);
}
```

for문이 한 번 루프될 때마다 cnt의 값이 1씩 증가하도록 합니다. 그러다 cnt 변수가 15에 도달하게 되면 for문이 0으로 초기화됩니다.(0~15=16자리 문자) 그리고 while문 안에 UCSR1A 함수를 통해 데이터레지스터가 비어있는지 확인하여 송신 준비를 합니다. 그리고 UDR1에 Tmp[cnt]는 순차적으로 증가하는 cnt의 숫자를 Tmp[] 배열 안에 넣어서 0, 1, 2, 3~15에 해당하는 각 문자들을 2초씩 딜레이를 갖고 출력합니다.

마지막 while(1)을 통해 프로그램이 종료되지 않도록 돕습니다.

### 추가실습3) AD 변환 값을 하이퍼터미널에 출력하시오.

```
#include <avr/io.h>
#include <util/delay.h>

unsigned char temp[3]={'0','.','0'}; // 변환값 표시를 위한 상수값 선언 및 초기화
int Result;
int i=0;
```

0.0의 배열을 갖는 temp 변수를 사용한다고 선언합니다. 그리고 Result와 I-0 변수도 사용합니다.

```
int main(void)
{

        PORTA=0x00;
```

```
TCCR0=0x6f;
```

메인함수를 시작하면서, 포트A를 초기화 하고 타이머/카운터 레지스터 0의 CTC mode, 분주비 1024를 사용하기로 합니다.

```
//--- ADC 초기 설정 ----------------------------------------
ADMUX=0x00;                          // 기준전압 → AREF(외부전압), 우측 정렬 저장,
                                     // Single ended input ADC0 채널 선택
ADCSRA=0b10010111;                   // ADC enable, Division Factor=128

while(1)
{
ADCSRA |=0x40;                       // AD 변환 시작: ADSC=1
while((ADCSRA&0x10)==0x00);          // AD 변환 완료까지 기다림. ADIF=1 check
```

ADC 기능을 사용하기 위해 초기 설정을 해줍니다. ADMUX를 통해 외부 전압을 기준 전압으로 사용하고, AD 변환 결과가 우측으로 정렬되게 합니다.
while 문을 열어서 ADCSRA |= 0x40;(=0b10010111;)을 통해 ADC 기능을 활성화하고, ADC가 변환되면 알려주는 플래그르 활성화합니다. 그리고 분주비를 128로 설정하여 ADC 최대 클럭 주파수보다 낮게 만들어 안정적으로 동작하게 합니다.

```
Result = ADC;                        // Result라는 변수에 ADC 변환 값 저장
_delay_ms(1);                        // 변환 시간 확보
Result=(Result>>2)*50/255;           // X.X로 표시하기위해 변환 결과를
                                     //  0~50으로 Scale
temp[0]=Result/10 + '0';             // 앞자리 LCD 출력 값 추출
temp[2]=Result%10 + '0';             // 소숫점 뒷자리 LCD 출력 값 추출
```

변환된 ADC값을 Result 변수에 저장하고 변환시간을 확보하기 위해 딜레이 0.001초를 줍니다. 그리고 0~표현되는 값을 0~50 범위로 줄이고, 그 줄인 Result 값을 몫과 나머지로 나누어서 [0].[0]~[5].[0]에 해당하는 temp[0]과 [2]에 대입합니다.

```
for(i=0; i<3; i++)
{
 _delay_ms(100);
```

for문을 통해 i 변수가 딱 3자리만 출력할 수 있도록 0, 1, 2까지만 작동하도록 합니다.
안정적으로 동작하기 위해 딜레이 함수를 넣습니다.

```
 UCSR1A=0x00;
 UCSR1B=0x18;                        // RXEN,TXEN set: 송수신기 enable
 UCSR1C=0x06;
 UBRR1L=0x33;                        // Baud Rate: 19.2k
```

```
UDR1=temp[i];                    // 수신 문자를 송신
}
```

이제 USART 기능에 관련된 레지스터들을 설정합니다. UCSR1B의 레지스터를 설정하여 수신기와 송신기를 활성화합니다. 나머지 설정은 추가실습1과 2와 같아 생략합니다.
UDR1에 for문에 따라 증가하는 i 변수를 통해 temp 문자 배열을 선택하여 출력하게 합니다.

```
if(i==3){
UDR1=0x0a;
_delay_ms(100);
UDR1=0x0d;
_delay_ms(100);
i=0;}
}
_delay_ms(1);

}
```

변수 I==3에 도달하면 UDR1에 줄바꿈 문자를 송신하게 하여 다음 줄로 바뀌게 하고, 캐리지 리턴 문자를 송신하여 커서를 맨 앞으로 위치시킵니다. 마지막으로 I = 0;을 하여 I가 초기화되게 합니다. UDR1에 송신 문자를 넣을때마다, 그리고 마지막에 딜레이 함수를 넣어 안정적으로 동작하게끔 합니다.

코드가 잘 작동하는지 확인하기 위해 가변저항을 소형 드라이버로 돌려가면서 ADC 값에 변화를 줍니다. 하이퍼 터미널 화면에 입력한 전압과 같은 전압이 출력되는지 확인합니다.

# 5. 측정값

**추가실습1) 수신 완료 인터럽트(Receive Complete interrupt)를 사용하여 키보드로 수신 받은 값을 하이퍼터미널로 송신하시오. (UCSR1B=0x98)**

| 키보드 입력 | 하이퍼 터미널 |
|---|---|
| 키보드 자판을 눌러서<br>사용자가 원하는 값을 입력함<br>ex) have a nice day | 키보드에서 입력했던 값이 그대로<br>하이퍼 터미널 프로그램에 출력됨<br>입력할 때마다 동작함.<br>ex) have a nice day |

표 2 ) 수신 완료 인터럽트를 사용한 USART 테스트 측정값

**추가실습2) 하이퍼터미널에 문자열을 출력하시오.**

| AVR Studio | 하이퍼 터미널 |
|---|---|
| "KNU_cj Jeong_PEL"을 문자로<br>출력할 수 있는 코드를 입력함 | "KNU_cj Jeong_PEL"라는 문자열이<br>문자열로 한 번에 출력되지 않고,<br>문자 하나씩 차례대로 출력됨.<br>한 번만 동작함. |

표 3 ) 원하는 문자열을 출력하는 USART 테스트 측정값

**추가실습3) AD 변환 값을 하이퍼터미널에 출력하시오.**

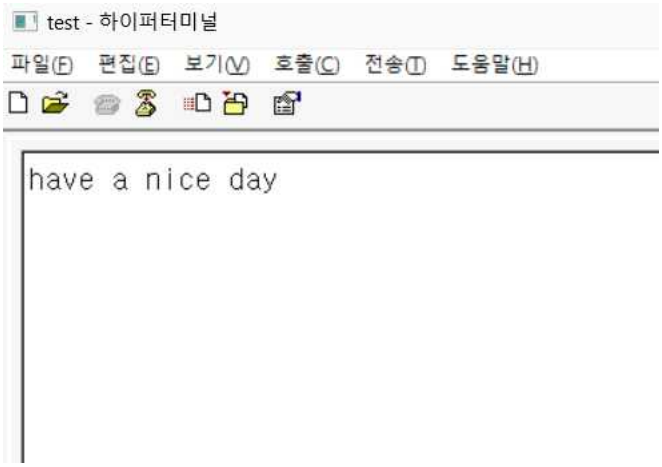
| AD 변환값 | 하이퍼 터미널 |
|---|---|
| 가변저항을 소형 드라이버로<br>돌려가면서 0.0V~5.0V 사이의<br>Analog 값을 입력함 | AD 변환값이 변화할 때마다 다음<br>줄에서 변화한 값으로 출력되는<br>동작을 반복함.<br>ex) 0.5 0.7 2.3 4.1 5.0 등등<br>(줄바꿈 포함) |

표 4 ) AD변환(가변저항)을 통한 USART 테스트 측정값

(실험결과가 한 페이지에 들어있으면 가독성이 좋을 것 같아 다음 페이지로 넘겼습니다.)

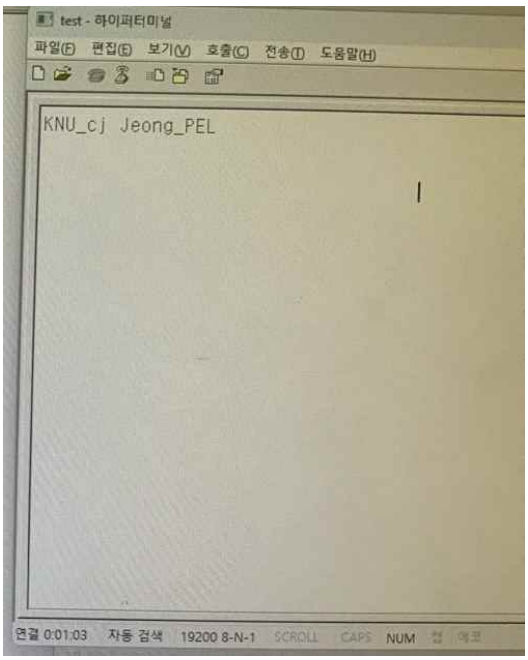
# 6. 실험결과

**추가실습1) 수신 완료 인터럽트(Receive Complete interrupt)를 사용하여 키보드로 수신 받은 값을 하이퍼터미널로 송신하시오. (UCSR1B=0x98)**



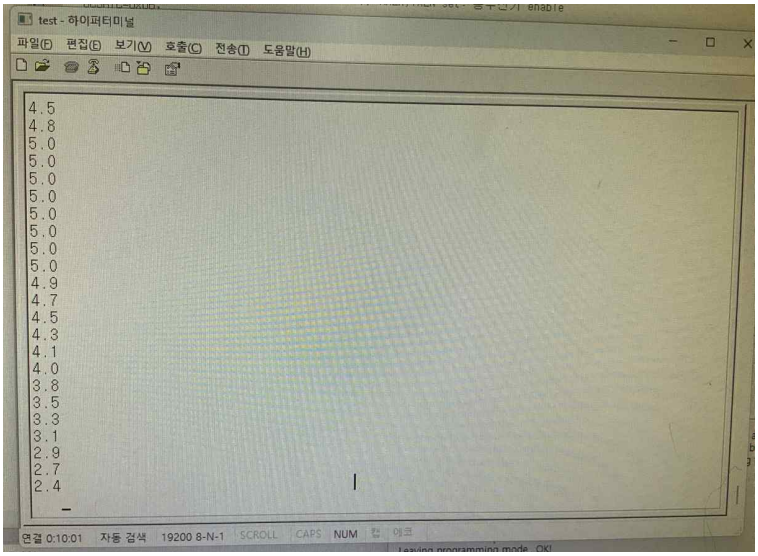

수신 완료 인터럽트를 사용하는 방법을 제대로 숙지하지 못하여 외부 인터럽트를 사용하는 ISR함수를 사용하였다가 실패하였다. 조교님께 질문 후 ISR 함수를 사용하는게 아니라고 알게 된 후 UCSR1A 레지스터의 RXC1 비트가 1이면 수신 데이터가 존재한다는 것을 이용하여 송수신을 하도록 만들었다.
정상적으로 키보드로 입력한 문자가 하이퍼 터미널 상에 출력되는 것을 정상적으로 확인하였다.

**추가실습2) 하이퍼터미널에 문자열을 출력하시오.**




문자열이 아닌 문자로 출력하려면 어떻게 해야 할까 고민하던 와중, 저번 타이머/카운터 실습에서 사용하였던 temp 문자 배열을 만들어 원하는 문자를 골라 출력하도록 하는 방식을 채택하였다. 문자열처럼 한 번에 출력되지 않고, 문자 하나하나 딜레이 2초마다 정상적으로 출력되는 것을 확인하였다.

**추가실습3) AD 변환 값을 하이퍼터미널에 출력하시오.**




가변저항을 소형 드라이버로 돌리며 ADC값을 조절하여도 하이퍼 터미널 상에서 제대로 변화하지 않았다. 첫 번째 문제점은 입력을 아무리 변화시켜도 하이퍼 터미널상 출력은 변화하지 않고, 0.0인 기본값만 출력되었다. 해결하기 위해 USART 관련 레지스터 함수들을 ADC while 문 안에 넣고 동작시켰더니 제대로 작동하였다. 두 번째 문제점은 입력이 1.5면 1.5가 출력되어야하는데 1. 으로 출력되는 문제가 있었다. for문에서 변수 I가 2보다 작을때까지만 동작하도록 하였더니 . 뒤에 나와야 할 숫자가 출력되지 않았던 것이다. 1을 높여 3보다 작게 작동하라고 만들어줬더니 정상적으로 출력되게 되었다.

# 7. 결과에 대한 논의

아쉬웠던 점은 지금까지 진행했던 실습 중에 보드 제작을 제외하고 비교적 어렵다고 느껴졌다. 각각의 레지스터 기능들을 숙지하고 난 뒤에는 수월하게 진행할 수 있었던 것을 생각하면, 레지스터들을 제대로 숙지하지 않았던 점이 문제였던 것 같았다. 추가실습2는 직접적인 문자 배열도 좋았지만, ASCII code를 사용하면 더 좋았을 것 같다는 생각이 든다.

좋았던 점은 temp 문자 배열 함수나 for과 while 같은 조건문을 사용하는 방법을 많이 알게 되었다. 어떤 상황에서 어떻게 넣어야 동작할지 조금씩 감이 잡히는 것 같아서 기쁘다. USART 기능을 통해 마이크로 컨트롤러 보드가 송수신을 통해 컴퓨터 상에 문자를 출력할 수 있다는게 참 대단하다고 느꼈다.

# 8. 결론

한 학기 동안 마이크로프로세서 응용 및 실험을 수강하면서 비동일계 편입생이지만 힘을 많이 얻었다. ATmega128 칩을 달은 조그만 보드 하나가 여러 가지 기능을 수행하는 것을 관찰하며 정말 다재다능하다고 느꼈다. 이 수업을 기점으로 코딩과 칩, 회로에 대한 지식을, 방학을 비롯하여 앞으로도 꾸준히 쌓아나가 이 조그맣지만 다재다능한 보드를 떠올리며 그 기억을 발판으로 삼고 전기전자공학 분야의 한 전문가로서 활동할 수 있도록 성장하겠다.

# 9. 참고문헌 및 출처

(교재) 김도인 교수님,「실습교재11」, 강원대학교, 전기전자공학과, (2~8p)
(교재) 김도인 교수님 [2024] 마프_Part 11_USART_ver1, "", (2~31p)
(사진) 이동제와 강민수, 실습 중 찍은 사진, 2024-06-13